

Language Modeling

Manu Madhavan

Department of Computer Science and Engineering,
Indian Institute of Information Technology Kottayam

Lecture 8

- Why probability to explain theory of language?
- Language Modeling
- Basic idea
- Examples
- N-gram Language Modeling

Language and cognition as probabilistic phenomena

- Human cognition is probabilistic and that language must therefore be probabilistic too since it is an integral part of cognition.
- We live in a world filled with uncertainty and incomplete information.
- We make decisions by processing these information
- These cognitive processes are best formalized as probabilistic processes
- Processing the words, forming an idea of the overall meaning of the sentence, and weighing it in making a decision is no different in principle
- Meaning of the word is defined by the circumstances of its use → uncertainty
- So Probability plays an important role in tackle the questions of meaning

- A language model is a probability distribution over strings on an alphabet

Language Modeling

- A language model is a probability distribution over strings on an alphabet
- Language Models are the development of probabilistic models that are able to predict the next word in the sequence given the words that precede it.

- A language model is a probability distribution over strings on an alphabet
- Language Models are the development of probabilistic models that are able to predict the next word in the sequence given the words that precede it.
- Language models explain various linguistics phenomena using probability
- It is a root problem for a large range of natural language processing tasks
 - Machine Translation
 - Spell checking
 - Next word prediction
 - Text generation

The cat

Example: Word Prediction



Example: Word Prediction



Example: Word Prediction



Example: Word Prediction



Language Model- Other Applications

- LM can assign probability to the sentence (how probable the sequences are?)

$$P(\textit{The cat sat on the mat}) = \\ P(\textit{The}).P(\textit{cat}|\textit{the}).P(\textit{sat}|\textit{the cat}).P(\textit{on}|\textit{The cat sat})...$$

- This will give higher probability to common/syntactically/semantically correct usages

Large wind tonight $\rightarrow 0.31$

High wind tonight $\rightarrow 0.97$

- Spelling mistake or correct word?

$P(I \text{ was late by fifteen minuets}) > P(I \text{ was late by fiteen minutes})$

- Speech Recognition

$P(I \text{ saw a van}) \gg P(\text{eyes awe of an})$

- Machine Translation: which usage is more plausible in target language

$P(\text{high wind}) > P(\text{large wind})$

$P(\text{He immediately moved to hospital}) >$

$P(\text{He quickly moved to hospital})$

- Spelling mistake or correct word?

$P(I \text{ was late by } \textit{fifteen minuets}) > P(I \text{ was late by } \textit{fiteen minutes})$

- Speech Recognition

$P(I \text{ saw a } \textit{van}) \gg P(\textit{eyes awe of an})$

- Machine Translation: which usage is more plausible in target language

$P(\textit{high wind}) > P(\textit{large wind})$

$P(\textit{He immediately moved to hospital}) >$

$P(\textit{He quickly moved to hospital})$

Language Model-Other Applications

- Context sensitive spelling correction
- Natural Language generation
- Next word prediction
- Auto completion (In mail, search engines)
 - Please, call
 - Can you

Language Model

- Goal: Compute the probability of a sentence or sequence of words
 $P(W) = P(w_1, w_2, \dots, w_n)$
- Related Task:
 $P(w_{n+1} | w_1, \dots, w_n)$
- Language model compute either of the above

- Probability of a sentence is the joint probability of the words

$$P(W) = P(w_1, w_2, \dots, w_n)$$

- This rely on conditional probability

$$P(A|B) = \frac{P(A,B)}{P(B)}$$

$$P(A, B) = P(B)P(A|B)$$

- This can be expanded

$$P(A, B, C, D) = P(A)P(B|A)P(C|A, B)P(D|A, B, C)$$

Counting Probabilities

- $P(\text{the cat sat on the mat}) = P(\text{the})P(\text{cat}|\text{the})P(\text{sat}|\text{the cat})\dots P(\text{mat}|\text{the cat sat on the})$
- How do you compute this probability?
$$P(\text{sat}|\text{the cat}) = \frac{\text{Count}(\text{the,cat,sat})}{\text{Count}(\text{the,cat})}$$
- We may never see enough data to estimate the long sequences

Markov Assumption and Language Model

- In the previous examples, the next word is represented by a conditional probability of previous word

$$P(W_{n+1}|W_1...W_n) = \frac{P(W_1...W_{n+1})}{P(W_1...W_n)}$$

- Exponential number of sequences, counting is difficult
- The question here is how much history is required?
 - It depends only a limited history (3 or 4 words)

- **Markov Assumption**

$$P(W_{n+1}|W_1...W_n) = P(W_{n+1}|W_n)$$

- Generally, we can say, it depends only on previous k words (k-order Markov model, $k < n$)

$$P(W_{n+1}|W_1...W_n) = P(W_{n+1}|W_{n-k}...W_n)$$

Markov Assumption and n-gram model

Looks only at n words at a time.

- 0^{th} order: $P(W_{n+1}|W_1...W_n) = P(W_{n+1})$ (independent of previous words, unigram model)
- 1^{st} order: $P(W_{n+1}|W_1...W_n) = P(W_{n+1}|W_n)$ (bigram model)
 $P(cat|the), P(sat|cat) ..$
- 2^{nd} order: $P(W_{n+1}|W_1...W_n) = P(W_{n+1}|W_n, W_{n-1})$ (trigram model)

Simple N-Gram Model

- Goal is to estimate the probability of a word given history, $P(w|h)$
- $P(\text{apple}|\text{the boy ate})$
- $P(\text{apple}|\text{the boy ate}) = \frac{\text{Count}(\text{the boy ate apple})}{\text{Count}(\text{the boy ate})}$
- Beware!— language is creative and new sentences are added every time
- It is hard to estimate sufficient statistics even from a large corpus (like web)

Some Notations

- Word sequence $w_1, \dots, w_n$ is represented as w_1^n
- Joint probability $P(X = w_1, Y = w_2, \dots)$ is represented as $P(w_1, w_2, \dots, w_n)$

Chain Rule

- $P(X_1, \dots, X_n) = P(X_1)P(X_2|X_1)P(X_3|X_1, X_2) \dots P(X_n|X_1, \dots, X_{n-1})$
- $P(X_1, \dots, X_n) = \prod_{i=1}^n P(X_i|X_1^{i-1})$
- $P(w_1^n) = \prod_{i=1}^n P(w_i|w_1^{i-1})$
- The joint probability of a sequence and computing the conditional probability of a word given previous words
- We don't know any way to compute the exact probability of a word given a long sequence of preceding words
- Approximate: The intuition of the N-gram model is that instead of computing the probability of a word given its entire history, we will approximate the history by just the last few words.

- Bigram model approximates the probability of a word given all the previous words
- $P(w_n | w_1^{n-1}) = P(w_n | w_{n-1})$
- $P(\text{apple} | \text{the boy ate}) = P(\text{apple} | \text{ate})$
- Markov Assumption
- We can generalize bigram to **N-gram** model

How to estimate N-gram probability

- Maximum Likelihood Estimate (MLE)
- For example to compute a particular bigram probability of a word y given a previous word x , we'll compute the count of the bigram $C(xy)$ and normalize by the sum of all the bigrams that share the same first word x
- $$P(w_n|w_{n-1}) = \frac{\text{Count}(w_{n-1}, w_n)}{\sum_w (\text{Count}(w_{n-1}, w))}$$
- Since the sum of all bigram counts that start with a given word w_{n-1} must be equal to the unigram count for that word w_{n-1}
- $$P(w_n|w_{n-1}) = \frac{\text{Count}(w_{n-1}, w_n)}{\text{Count}(w_{n-1})}$$

Example

<s> I am Sam </s>

<s> Sam I am </s>

<s> I do not like green eggs and ham </s>

Here are the calculations for some of the bigram probabilities from this corpus

$$P(I | <s>) = \frac{2}{3} = .67 \quad P(\text{Sam} | <s>) = \frac{1}{3} = .33 \quad P(\text{am} | I) = \frac{2}{3} = .67$$

$$P(</s> | \text{Sam}) = \frac{1}{2} = 0.5 \quad P(\text{Sam} | \text{am}) = \frac{1}{2} = .5 \quad P(\text{do} | I) = \frac{1}{3} = .33$$

- For N-gram model: $P(w_n | w_1^{n-1}) = \frac{\text{count}(w_{n-N+1}^{n-1} w_n)}{\text{Count}(w_{n-N+1}^{n-1})}$
- This ratio is called relative frequency

Example

	i	want	to	eat	chinese	food	lunch	spend
i	5	827	0	9	0	0	0	2
want	2	0	608	1	6	6	5	1
to	2	0	4	686	2	0	6	211
eat	0	0	2	0	16	2	42	0
chinese	1	0	0	0	0	82	1	0
food	15	0	15	0	1	4	0	0
lunch	2	0	0	0	0	1	0	0
spend	1	0	1	0	0	0	0	0

Figure 4.1 Bigram counts for eight of the words (out of $V = 1446$) in the Berkeley Restaurant Project corpus of 9332 sentences.

Fig. 4.2 shows the bigram probabilities after normalization (dividing each row by the following unigram counts):

i	want	to	eat	chinese	food	lunch	spend
2533	927	2417	746	158	1093	341	278

Example

$$P(i | <s>) = 0.25$$

$$P(\text{food} | \text{english}) = 0.5$$

$$P(\text{english} | \text{want}) = 0.0011$$

$$P(</s> | \text{food}) = 0.68$$

Example

	i	want	to	eat	chinese	food	lunch	spend
i	0.002	0.33	0	0.0036	0	0	0	0.00079
want	0.0022	0	0.66	0.0011	0.0065	0.0065	0.0054	0.0011
to	0.00083	0	0.0017	0.28	0.00083	0	0.0025	0.087
eat	0	0	0.0027	0	0.021	0.0027	0.056	0
chinese	0.0063	0	0	0	0	0.52	0.0063	0
food	0.014	0	0.014	0	0.00092	0.0037	0	0
lunch	0.0059	0	0	0	0	0.0029	0	0
spend	0.0036	0	0.0036	0	0	0	0	0

Figure 4.2 Bigram probabilities for eight words in the Berkeley Restaurant Project corpus of 9332 sentences.

$$\begin{aligned}P(<s> \text{ i want english food } </s>) \\&= P(i|<s>)P(\text{want}|i)P(\text{english}|\text{want}) \\&\quad P(\text{food}|\text{english})P(</s>|\text{food}) \\&= .25 \times .33 \times .0011 \times 0.5 \times 0.68 \\&= .000031\end{aligned}$$

So far

- Language Modeling is assigning probabilities to sequence of words/characters
- Markov assumption and N-gram models
- Counting probability
- Example

Next

- Evaluation - Perplexity
- Smoothing

Training and Test corpus

- The probabilities of the N-gram model come from the corpus it trained on
- Training on some data and testing on some other data (mutually exclusive sets)
- Divide the dataset into **Training corpus and test corpus**
- Its important not to let the test sentences into the training set

Training and Test corpus

- The probabilities of the N-gram model come from the corpus it trained on
- Training on some data and testing on some other data (mutually exclusive sets)
- Divide the dataset into Training corpus and test corpus
- **Its important not to let the test sentences into the training set**
- How to split input corpus: 80:20, 70:30,...

Issues in N-gram modeling

- The probabilities of the N-gram model come from the corpus it trained on
- It may not TRUE for another testing corpus
- Sensitivity to the value of N
- Create a general purpose corpus by collecting different genres

Treating unknown words

- Test set may contain unknown words (Out of Vocabulary words-OOV)
- Choose a Vocabulary which is fixed
- Convert OOV to $\langle UNK \rangle$ unknown category
- Estimate the probability of $\langle UNNK \rangle$

Evaluating the model

- **Extrinsic** (in vivo) vs intrinsic evaluation
- If know the expected model in prior, we can use extrinsic measures (eg: accuracy)
- For example: We can apply a model on some application (say text classification and predict accuracy)
- **Intrinsic** measures evaluate without any external applications
- Perplexity

- **Perplexity** is the most common intrinsic evaluation metric for N-gram language models.
- Given two probabilistic models, **the better model is the one that has a tighter fit to the test data**, or predicts the details of the test data better.
- The perplexity (sometimes called PP for short) of a language model on a test set is a function of the probability that the language model assigns to that test set.
- For sequence of $W = w_1, \dots, w_N$
- $PP(W) = P(w_1, \dots, w_N)^{-1} = \sqrt[N]{\frac{1}{P(w_1, \dots, w_N)}}$

Perplexity

- For sequence of $W = w_1, \dots w_N$
- $PP(W) = P(w_1, \dots w_N)^{-1} = \sqrt[N]{\frac{1}{P(w_1, \dots w_N)}}$
- $PP(W) = \sqrt[N]{\prod_{i=1}^n \frac{1}{P(w_i | w_1, \dots w_{i-1})}}$
- $PP(W) = \sqrt[N]{\prod_{i=1}^n \frac{1}{P(w_i | w_{i-1})}}$ For bigram model

Perplexity



- The best model will be the one that assigns the highest probability to the test set.
- Intuitively, if a model assigns a high probability to the test set, it means that it is not surprised to see it (its not perplexed by it)
- Low perplexity indicates it has a good understanding of how the language works.
- $PP(W) = \sqrt[n]{\prod_{i=1}^n \frac{1}{P(w_i|w_1,..w_{i-1})}}$ (we are taking inverse of probability)
- A lower perplexity indicates a better model

- Perplexity also indicate average branching factor of a language model
- Branching factor is the number of possible next words that can follow a word
- perplexity is high means: *“when trying to guess the next word, our model is as confused as if it had to pick between many different words”*

Next: Smoothing

- The estimating probabilities from sparse data
- Zero count affect entire MLE computation
- Addressing poor estimation: smoothing
- Different Smoothing techniques
 - Laplace smooting
 - Good-Turing discounting
- Interpolation

Read: Chapter 4, SLP, Jurafsky